

# LevelSyn: Physical-Aware Logic Synthesis via Level-Asynchronous Graph Neural Networks

Jingyi Zhou, Zhengyuan Shi\*, Ziyang Zheng, Qiang Xu\*

The Chinese University of Hong Kong, Hong Kong, China

{jyzhou26, zyshi21, zyzheng23, qxu}@cse.cuhk.edu.hk

## Abstract

As integrated circuit technology scales into the nanometer regime, the traditional disconnect between logic synthesis and physical design has led to significant PPA (Power, Performance, and Area) degradation and prolonged design closure cycles. Traditional logic synthesis relies on non-physical Wire Load Models (WLMs), while recent spectral-based placement predictors often neglect the inherent hierarchical logic depth and signal flow of netlists, which leads to low-fidelity spatial estimations. To bridge this gap, we propose LevelSyn, a novel physical-aware logic synthesis framework that integrates hierarchical representation learning with a wirelength-driven optimization engine. At its core, LevelSyn leverages a level-asynchronous Graph Neural Network (GNN) to predict high-fidelity gate coordinates by capturing the structural and directional semantics of And-Inverter Graphs (AIGs). To handle industrial-scale designs, a level-aligned subgraph partitioning strategy is introduced to eliminate memory bottlenecks while preserving local logical dependencies. These spatial insights are seamlessly integrated into a newly developed physical-informed synthesis engine within the Berkeley ABC framework. Experimental results on the EPFL benchmark suite demonstrate that LevelSyn significantly outperforms state-of-the-art (SOTA) methods, achieving an average power reduction of 6.89% and a timing delay improvement of 27.48%. Furthermore, post-place-and-route validation shows a 99.59% reduction in design rule check (DRC) violations, highlighting its effectiveness in accelerating design convergence.

## 1 Introduction

To avoid local optimization and time-consuming design iterations within the current stage-based chip design workflow, the Electronic Design Automation (EDA) industry has actively adopted a “shift-left” strategy [5, 35]. The future design metrics and potential violations should be exposed at much early stages, so that engineers can make revisions on the “left” of the workflow.

Physical-aware synthesis [25] is a prominent example in logic synthesis domain. Unlike conventional tools [23, 34] estimate the design quality based on models only considering the logical abstraction (e.g., wire load models), Physical-aware synthesis tries

to combine the backend information to enable accurate estimation of interconnect delay and congestion during early stages. Although logic synthesis can produce promising results, they are often poorly suited for physical implementation in modern nanometer-scale designs [13, 22]. To bridge this gap, the early attempts explored the physical-aware synthesis, which incorporates physical information into the early stages of logic synthesis, thereby closing the gap between logical abstraction and physical implementation [3, 10, 11, 26, 27].

Most recently, graph analysis techniques have been employed to predict physical metrics. For example, Net<sup>2</sup>[37] and another attempt [36] leverage Graph Neural Networks (GNNs) for pre-placement wirelength estimation. CPA-Remap [12] identifies the critical paths and merges and then remaps the netlist for timing optimization. However, these predictive models are tied to a specific circuit structure. As the logic synthesis constantly reshapes the logic network, these methods are limited to minor refinement after synthesis. Other frameworks [12, 16, 24] combine the fast physical estimation [21] into logic optimization and technology mapping tools. Although they can predict the connectivity and spatial distance between logic cells, other critical information, such as hierarchical level structure and directional signal flow, are neglecting. Therefore, efficiently estimating high-fidelity physical information for logic synthesis remains a significant challenge.

In this work, we propose LevelSyn, a physical-aware logic synthesis framework that leverages a hierarchical neural network to estimate spatial coordinates and orientations, bridging the gap between logical abstraction and physical implementation. Specifically, our framework leverages a single-round and level-asynchronous GNN (LA-GNN) architecture inspired by DeepGate2 [29]. We propose novel mechanism to encode and propagate the pre-defined constraints of Primary Inputs (PIs) and Primary Outputs (POs). Unlike existing physical-aware synthesis methods, LevelSyn produces row-consistent spatial priors. In this approach, the cell orientation is directly derived from the predicted  $y$ -coordinates based on the row specifications in the PDK. This ensures that each cell aligns correctly with the power rails without additional processing. Moreover, to ensure the scalability for industrial-scale designs, we employ a level-aligned subgraph partitioning strategy that mitigates memory overhead.

We implement our model into an open-source logic synthesis tool ABC [23]. Our model provides the spatial insights of the given pre-mapping logic and guides the technology mapping engine by redefining its cost function. The enhanced ABC tool produces the post-synthesize circuit netlists that are inherently optimized for physical implementation. Using the DREAMPlace [19] for physical design validation, the experimental results on the EPFL benchmark suite demonstrate that LevelSyn outperforms state-of-the-art

Our code is available at <https://github.com/zhoujy22/PhysicalAwareSynthesis>.

\*Corresponding authors



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3833940>

methods, achieving an average power reduction by 6.89% and a timing delay reduction by 27.48%. Furthermore, post-place-and-route validation reveals a 99.59% reduction in design rule check (DRC) violations, highlighting the framework's effectiveness in generating layout-friendly netlists and accelerating design convergence for advanced chip products.

The contributions of this work is summarized as follows:

- We propose a predictive engine based on the Level-Asynchronous GNN (LA-GNN) that explicitly captures the directional signal flow and hierarchical logic depth of And-Inverter Graphs (AIGs). To ensure scalability for industrial-scale designs, we introduce a Level-Aligned Subgraph Partitioning (LASP) strategy that mitigates memory bottlenecks while preserving critical local logical dependencies.
- We develop a high-fidelity spatial prediction scheme that estimates gate coordinates and captures the underlying standard-cell row structure. By leveraging these predictions, the framework derives deterministic priors for cell orientation and power rail alignment, providing more comprehensive physical guidance than traditional spectral-based methods.
- We provide LevelSyn, an end-to-end framework combining GNN with open-source logic synthesis tool. By substituting traditional heuristics with a physical-informed cost function, our framework achieves better performance, lower power consumption and lower post-routing DRC violations within the acceptable runtime.

## 2 Related Works

### 2.1 Evolution of Physical Aware Synthesis

Traditionally, VLSI design flows maintained a strict separation between front-end logic synthesis and back-end physical design. However, as semiconductor technology scaled into the deep sub-micron (DSM) era, interconnect delays began to dominate gate delays, rendering traditional logic-depth metrics insufficient for timing closure [13].

To bridge this gap, early research explored the integration of logic optimization and placement. Gosti et al. [11] addressed timing closure by interleaving logic operations with global placement, while simultaneous technology mapping and placement algorithms like Maple [32, 33] sought to incorporate spatial information directly into the mapping process. Despite these efforts, early techniques often suffered from high computational complexity or relied on inaccurate wire-load models (WLM) that failed to capture the nuances of modern routing congestion. Recent industry shifts toward frameworks like OpenROAD [31] provide better infrastructure for integration, yet achieving a truly unified metric remains a challenge.

The PigMap series [16, 24] introduced a paradigm shift by utilizing primitive logic gate placement to guide technology mapping. PigMap1 extracts high-fidelity spatial information, specifically estimated coordinates, from a primitive placement of technology-independent gates. This spatial prior allows the mapper to optimize for performance (minimizing critical path Manhattan distance) or power (reducing total wirelength). PigMap2 further refined this by incorporating multi-dimensional metrics such as congestion-aware placement. While successful, these methods rely on fast placement engines that often yields inaccurate spatial data, creating a fidelity

gap that prevents the subsequent mapping phase from achieving truly optimal results.

### 2.2 GNNs in EDA and Their Limitations

With the rise of machine learning, Graph Neural Networks (GNNs) have become a popular choice for EDA tasks due to the natural representation of netlists as graphs [4]. Extensive research has focused on learning structural and functional representations for combinational circuits. For instance, DeepGate [17] utilizes attention-based aggregation and skip connections to encode Boolean computations, benefiting downstream tasks such as test point insertion [28] and Boolean Satisfiability (SAT) [18]. This was further evolved by DeepGate2 [29], which introduced pairwise truth-table supervision and functionality-aware loss functions to enhance scalability. Other approaches have explored contrastive learning to derive invariant representations, such as FGNN [7] for circuit classification, as well as CircuitFusion [9] and NetTAG [8] for multi-modal circuit data. Furthermore, Polargate [20] enhanced representation quality by employing the bipolar principle.

Despite these advancements in encoding logical functionality, applying standard GNN architectures to large-scale physical-aware logic synthesis presents significant practical hurdles. First, the primary bottleneck lies in memory consumption. Modern netlists containing millions of gates require massive graph structures and message-passing buffers that frequently exceed the capacity of standard GPU hardware [29]. Second, the iterative nature of graph convolutions introduces a substantial runtime overhead compared to traditional heuristics. In a high-speed synthesis environment where rapid design iterations are essential, the inference latency of complex GNNs often outweighs their accuracy benefits [14]. Most importantly, while existing representation learning excels at capturing Boolean behavior, it does not natively provide the explicit spatial coordinates required for physical optimization. Therefore, there is a growing need for more efficient neural architectures that can directly provide accurate spatial priors without the prohibitive computational and memory costs associated with standard GNN-based representation learning.

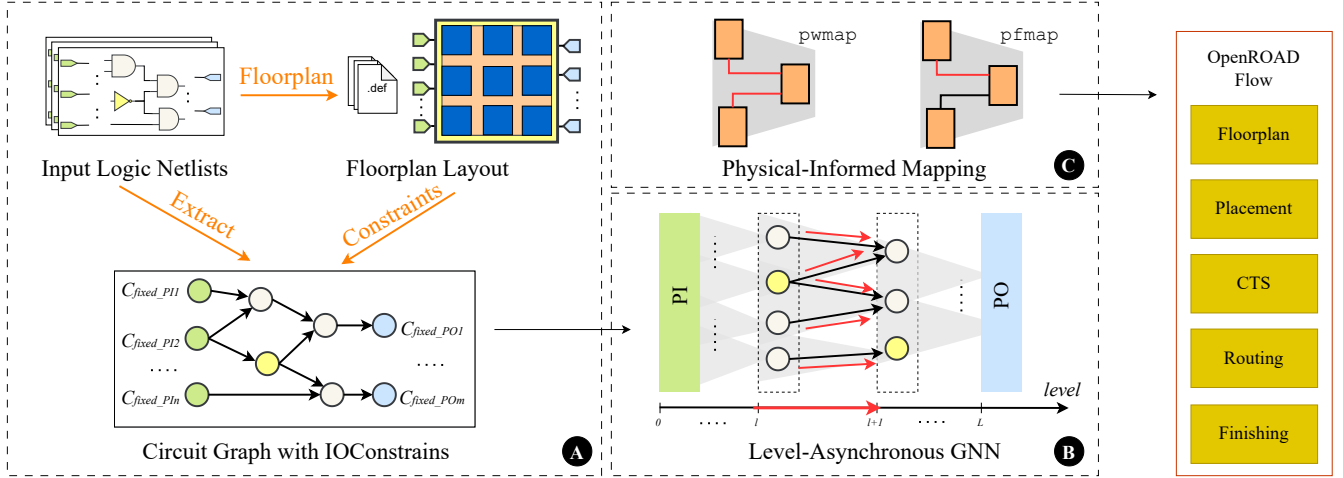
## 3 Methodology

### 3.1 Overview of the Proposed Framework

The LevelSyn framework is an end-to-end, physical-aware logic synthesis flow designed to bridge the gap between logical abstraction and physical implementation. As illustrated in Fig. 1, the framework consists of three main phases: Hierarchical Pre-processing, Scalable Spatial Prediction, and Physical-Informed Technology Mapping.

**3.1.1 Problem Formulation.** We represent the input logic netlist as an And-Inverter Graph (AIG), denoted by a directed acyclic graph  $G = (V, E)$ . Each node  $v \in V$  represents a logic gate (2-input AND or Inverter), and each edge  $e \in E$  represents the signal flow. Given the fixed boundary constraints from the floorplan stage, specifically the spatial coordinates and the orientations of primary inputs and outputs (PI/POs), denoted as  $C_{fixed} = \{(x_i, y_i, O_i) \mid i \in PI \cup PO\}$ . Our objective is twofold:

- Predict high-fidelity spatial coordinates and capture orientations  $(x_v, y_v, O_v)$  for all internal gates  $v \in V \setminus (PI \cup PO)$ .



**Figure 1: Overview of LevelSyn framework. (A) AIG floorplan to obtain IO constraints. (B) Cell spatial coordinates inference with layer-asynchronous GNN. (C) Physical-informed mapping technique to generate post-mapping netlists.**

- Optimize the technology mapping process by utilizing these coordinates to minimize post-routing wirelength and delay.

**3.1.2 Framework Pipeline.** The execution flow of LevelSyn follows a structured sequence:

**Hierarchical Pre-processing.** The framework first parses the AIG to extract topological features and hierarchical level information. To ensure global spatial awareness, fixed PI/PO coordinates are encoded as spatial anchors. For industrial-scale designs, a level-aligned subgraph partitioning strategy is applied to decompose the flattened graph into manageable segments, preventing memory overflow while maintaining logical continuity.

**Scalable Spatial Prediction.** The partitioned AIG segments are fed into the Level-Asynchronous GNN. Unlike standard GNNs that perform undirected message passing, our engine propagates features along the directional signal flow, which could capture the correlation between logic depth and physical placement. The output is a set of spatial priors ( $\hat{x}_v, \hat{y}_v, \hat{o}_v$ ) for every gate in the netlist.

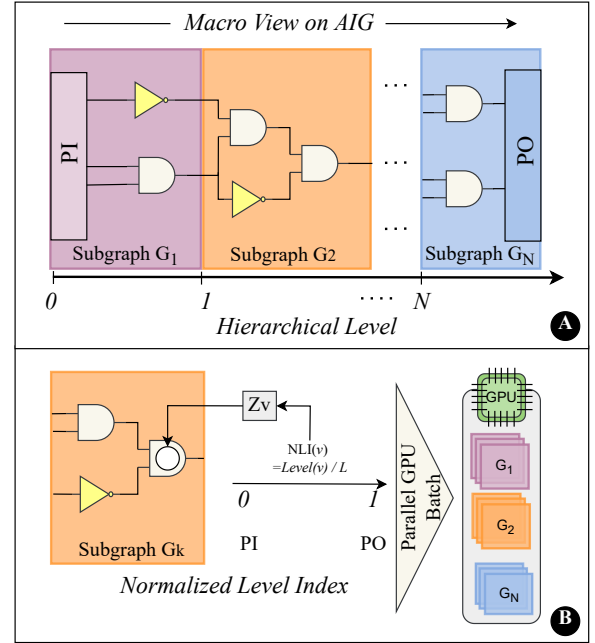
**Physical-Aware Technology Mapping.** These spatial priors are integrated into the Berkeley ABC engine. By extending the mapping cost function to include wirelength-driven metrics, LevelSyn enables the synthesis engine to make physical-aware choices during gate-level optimization, resulting in a netlist with superior PPA.

### 3.2 Level-Aware GNN Architecture

The core of our predictive engine is a Level-Aware Graph Neural Network (LA-GNN) designed to map the logical structure of an AIG to high-fidelity spatial priors. LA-GNN explicitly respects the directional signal flow and hierarchical levels of the netlist.

**3.2.1 Level-Asynchronous Message Passing.** To capture the logic depth and signal propagation, we first partition the AIG nodes into sequential levels  $\{L_1, L_2, \dots, L_D\}$ , where  $D$  is the maximum logic depth. A node  $v$  belongs to level  $L_k$  if the maximum path length from any PI to  $v$  is  $k$ .

The initial feature vector  $z_v$  for each node  $v \in V$  is constructed by concatenating structural, functional, and spatial attributes to



**Figure 2: Subgraph partitioning scheme. (A) Level-aligned subgraph partitioning overview. (B) Global positional priors injection and parallel GPU batch processing**

provide a rich starting point for the level-asynchronous message passing. Specifically,  $z_v$  is defined as:

$$z_v = [T_v \parallel S_v \parallel D_v] \quad (1)$$

where  $\parallel$  denotes the concatenation operator. The components are defined as follows:

**Gate Type Encoding ( $T_v$ ):** We use a one-hot encoding vector to represent the functional type of the node.

**Spatial Anchors ( $S_v$ ):** This is a critical feature for shift-left physical awareness. For nodes  $v \in PI \cup PO$ ,  $S_v$  contains their fixed

$(x, y)$  coordinates obtained from the floorplan stage, normalized to the range  $[0, 1]$ . For internal logic gates whose locations are unknown,  $S_v$  is initialized with the center of the layout  $(0.5, 0.5)$  for a place-holder value. This allows the GNN to treat PI/POs as spatial anchors that guide the coordinate regression of internal nodes.

**Structural Attributes ( $D_v$ ):** To capture the local complexity of the netlist, we include the fan-in and fan-out degrees of each node. Additionally, the pre-calculated topological level index is embedded as a numerical feature, helping the GNN perceive its relative depth within the entire signal path.

To ensure that every internal gate perceives the spatial constraints from both PIs and POs, we propose a Bi-directional Level-Asynchronous Message Passing mechanism. This approach consists of a forward pass to capture signal flow and a backward pass to diffuse spatial constraints from the outputs.

**Forward Signal Propagation.** We first propagate features along the topological order from level  $L_1$  to  $L_D$ . For a node  $v \in L_k$ , the forward hidden state  $\vec{h}_v^{(k)}$  captures the logical influence from its fan-in:

$$\vec{m}_v^{(k)} = \text{AGGREGATE} \left( \vec{h}_u^{(j)} \mid u \in \text{FI}(v), j < k \right) \quad (2)$$

$$\vec{h}_v^{(k)} = \sigma \left( \text{Wfwd} \cdot \text{CONCAT}(\vec{m}_v^{(k)}, z_v) \right) \quad (3)$$

**Backward Constraint Diffusion.** To incorporate PO spatial constraints, a second pass is performed in reverse topological order ( $L_D$  to  $L_1$ ). For a node  $v \in L_k$ , its backward hidden state  $\overleftarrow{h}_v^{(k)}$  is updated by its fan-out nodes (FO):

$$\overleftarrow{m}_v^{(k)} = \text{AGGREGATE} \left( \overleftarrow{h}_u^{(j)} \mid u \in \text{FO}(v), j > k \right) \quad (4)$$

$$\overleftarrow{h}_v^{(k)} = \sigma \left( \text{Wbwd} \cdot \text{CONCAT}(\overleftarrow{m}_v^{(k)}, z_v) \right) \quad (5)$$

**Final Embedding Fusion.** The final high-dimensional embedding  $h_v^*$  is the concatenation of both directional states:

$$h_v^* = [\vec{h}_v^{(k)} \parallel \overleftarrow{h}_v^{(k)}] \quad (6)$$

This dual-flow architecture ensures that the structural representation of each gate is constrained by PI and PO anchors, allowing the downstream MLP to interpolate coordinates that respect the global floorplan.

**3.2.2 Orientation Estimation.** In a standard-cell-based layout, a cell's orientation is not an independent variable but is often constrained by its vertical position relative to the power rails. We derive the orientation estimation  $\hat{O}_v$  based on the predicted  $\hat{y}_v$  and the row structure defined in the floorplan. First, we map the predicted  $\hat{y}_v$  to a discrete row index  $R_v$ :

$$R_v = \text{round} \left( \frac{\hat{y}_v - y_{\text{offset}}}{H_{\text{row}}} \right) \quad (7)$$

where  $H_{\text{row}}$  is the standard cell row height and  $y_{\text{offset}}$  is the layout boundary. Given the power rail sharing property (VDD-VSS-VSS-VDD), the orientation  $\hat{O}_v$  is estimated as:

$$\hat{O}_v = \begin{cases} \text{N}, & \text{if } R_v \bmod 2 = 0 \\ \text{FS}, & \text{if } R_v \bmod 2 = 1 \end{cases} \quad (8)$$

This mechanism ensures that the predicted spatial priors  $(\hat{x}_v, \hat{y}_v, \hat{O}_v)$  are physically consistent with the target layout environment, allowing the synthesis engine to better estimate the pin locations and parasitic effects during the mapping phase.

### 3.3 Scalable Subgraph Partitioning

To ensure the scalability of LevelSyn on modern designs while preserving the directional logic dependencies, we propose a Level-Aligned Subgraph Partitioning (LASP) strategy (Figure2).

Unlike traditional graph partitioning algorithms [6, 15] that focus on minimizing edge cuts but often disrupt the topological flow, our LASP strategy respects the hierarchical levels of the AIG. Given an AIG with a maximum logic depth  $L$ , we divide the graph into  $N$  contiguous segments along the level dimension. Each subgraph  $G_k = (V_k, E_k)$  is defined as:

$$V_k = \{v \in V \mid (k-1) \cdot w \leq \text{Level}(v) < k \cdot w\} \quad (9)$$

where  $w = \lceil L/N \rceil$  is the level window size. This partitioning ensures that each subgraph contains a complete "logic slice" of the circuit, maintaining the local connectivity between adjacent logic levels which is critical for spatial proximity estimation.

A key challenge in independent subgraph partitioning is the loss of global coordinates. Instead of using hardware-inefficient sequential propagation, LevelSyn injects Global Positional Priors directly into the initial node features  $z_v$ . We define a Normalized Level Index (NLI) for each node:

$$\text{NLI}(v) = \frac{\text{Level}(v)}{L} \quad (10)$$

The NLI serves as a global GPS for the GNN, allowing each subgraph—processed in parallel within a GPU batch to perceive its relative distance between the PI (NLI  $\approx 0$ ) and PO (NLI  $\approx 1$ ) boundaries. By combining NLI with the fixed PI/PO spatial anchors, the model can maintain a globally consistent coordinate system.

By controlling the window size  $w$ , the memory complexity of each GNN pass is reduced from  $O(|V|)$  to  $O(|V|/N)$ , effectively capping the peak memory usage regardless of the total circuit size. This enables LevelSyn to handle industrial benchmarks that are orders of magnitude larger than those typically supported by spectral-based methods, which often suffer from the cubic complexity of Laplacian eigen-decomposition.

### 3.4 Physical-Aware Technology Mapping

Figure 3 illustrates the final stage of the LevelSyn framework. It integrates the predicted spatial priors into the technology mapping process within the Berkeley ABC [2] engine. Drawing inspiration from the physical-aware synthesis paradigm established by PigMAP [24], we bridge the gap between logical structural optimization and physical layout requirements.

**3.4.1 Physical Data Integration.** To enable physical awareness, the predicted coordinates  $(\hat{x}_v, \hat{y}_v)$  and orientations  $\hat{O}_v$  for every AIG node are back-annotated into the ABC data structure. We extend the AIG node representation in ABC to include a physical attribute vector  $\mathcal{P}_v = (\hat{x}_v, \hat{y}_v, \hat{O}_v)$ . During the mapping phase, these attributes are used to estimate the wirelength of potential subject graphs (matches) before they are committed to the final netlist.

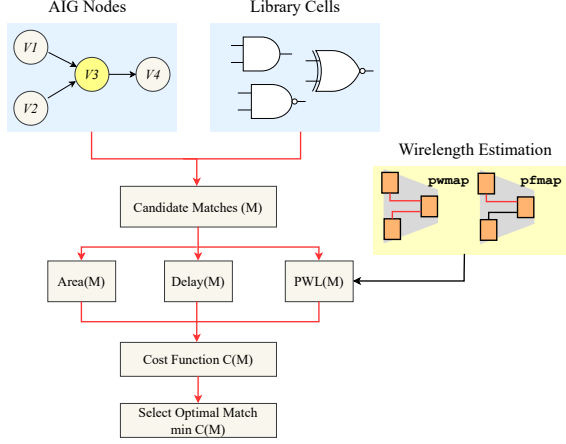


Figure 3: Explanation of the technology mapping flow

**3.4.2 Wirelength-Driven Cost Function.** In standard technology mapping, the engine selects a library gate (cell) to cover a portion of the AIG by minimizing a cost function typically focused on area or arrival time. LevelSyn redefines this cost function to include a Predicted Wirelength (PWL) metric. Following the differentiated cost evaluation strategy proposed in PigMAP [24], we adapt the PWL calculation based on the specific optimization target to better reflect the underlying physical impact. For a candidate match  $M$  that implements a set of AIG nodes, the augmented cost  $C(M)$  is defined as:

$$C(M) = \alpha \cdot \text{Area}(M) + \beta \cdot \text{Delay}(M) + \gamma \cdot \text{PWL}(M) \quad (11)$$

where  $\alpha, \beta, \gamma$  are weighting factors. The  $\text{PWL}(M)$  is estimated using the Manhattan distance between the predicted center of the candidate cell  $v_M$  and its fan-in/fan-out neighbors. For power-oriented command **pwmap**, we calculated the sum of Manhattan distance to all adjacent pins to minimize total switching power:

$$\text{PWL}(M) = \sum_{u \in \text{adj}(v_M)} (|\hat{x}_{v_M} - \hat{x}_u| + |\hat{y}_{v_M} - \hat{y}_u|) \quad (12)$$

Conversely, for delay-oriented mapping command **pfmap**, we employ the maximum Manhattan distance to bound the worst-case interconnect delay on critical paths:

$$\text{PWL}(M) = \max_{u \in \text{adj}(v_M)} (|\hat{x}_{v_M} - \hat{x}_u| + |\hat{y}_{v_M} - \hat{y}_u|) \quad (13)$$

**3.4.3 Mapping Execution and Optimization.** The execution of the physical-informed mapping is implemented as an enhanced flow within the Berkeley ABC engine, extending standard mapping commands such as **map**, **pfmap**, and **pwmap**. During this process, the engine first identifies all potential library cell matches for the subject AIG and evaluates them using the augmented cost function described in Section 3.4.2. By prioritizing matches that align with the predicted spatial trajectory, in particular those that minimize net stretching relative to the fixed PI/PO anchors, the engine ensures a strong correlation between logical structures and physical constraints. This is followed by an iterative refinement phase where LevelSyn performs local gate-level swapping and sizing; a gate is

replaced if a functionally equivalent library cell provides a superior fit to the predicted spatial prior, thereby reducing estimated wirelength without violating timing constraints. By embedding physical location directly into the mapping decision tree, LevelSyn generates a netlist that is inherently layout-friendly, significantly mitigating routing congestion and wire-induced delay in the subsequent physical design stages.

## 4 Experiments

In this section, we evaluate the performance of LevelSyn against industry-standard baselines and SOTA physical-aware synthesis methods. We focus on the framework’s ability to optimize PPA (Power, Performance, and Area) metrics across a diverse set of benchmarks.

### 4.1 Experimental Settings

**Benchmark Suite.** We utilize the EPFL Combinational Benchmark Suite [1] to assess scalability. The circuit sizes range from hundreds of nodes to over 200k nodes.

**EDA Flow.** All designs are processed using Berkeley ABC for logic synthesis and technology mapping. For physical ground truth and spatial anchor generation, we employ DREAMPlace [19] to obtain high-quality placement coordinates. The target library is the SkyWater Open Source PDK [30] standard cell library.

**GNN Training Configuration.** To ensure the reproducibility of our LA-GNN model, we detail the training environment and hyperparameters as follows: We employ a multi-task loss function  $\mathcal{L} = \mathcal{L}_{MSE}(\hat{x}, \hat{y}) + \mathcal{L}_{BCE}(\hat{O})$  to simultaneously optimize coordinate regression and orientation prediction.  $\mathcal{L}_{MSE}$  denotes the Mean Squared Error for the predicted coordinates, while  $\mathcal{L}_{BCE}$  represents the Binary Cross-Entropy loss for orientation classification. The model is trained using the AdamW optimizer with an initial learning rate of  $1 \times 10^{-3}$  and a weight decay of  $1 \times 10^{-4}$ . All computational tasks are performed on a Linux workstation equipped with a NVIDIA A100 GPU. We use a hidden dimension of 128 and 6 message-passing layers. The partition window size  $w$  is set to 5000 nodes.

**Baselines.** We employ both the industry-standard baselines and SOTA synthesis methods as baselines, and we address them as follows:

- **Baseline:** Standard logic-driven mapping in ABC (**map**).
- **PigMap-power/performance** [24]: The SOTA physical-aware mapping that uses analytical wireload models for power and delay optimization.

### 4.2 GNN Pre-training and Fine-tuning Results

To enhance the model’s ability to capture generalized structural representations of logic circuits, we adopt a two-stage training strategy consisting of large-scale pre-training followed by task-specific fine-tuning. In the pre-training phase, we construct a diverse dataset comprising 500 to 10,000 randomly generated AIGs with varying topological complexities, which allows the LA-GNN to learn intrinsic geometric and logical features across a broad synthetic design space. Subsequently, the pre-trained weights are used as an initialization for the model, which is then fine-tuned on specific circuits from established benchmarks to adapt the learned representations

**Table 1: PPA Comparison under Different Technology Mapping Strategies**

| Circuit    | Nodes   | Baseline               |              |           | PigMap-power           |              |           | PigMap-performance     |              |           |
|------------|---------|------------------------|--------------|-----------|------------------------|--------------|-----------|------------------------|--------------|-----------|
|            |         | Area(um <sup>2</sup> ) | Delay(ps)    | Power(mW) | Area(um <sup>2</sup> ) | Delay(ps)    | Power(mW) | Area(um <sup>2</sup> ) | Delay(ps)    | Power(mW) |
| ctrl       | 181     | 571.80                 | 760.23       | 0.0063    | 604.33                 | 737.59       | 0.0089    | 539.27                 | 826.13       | 0.0078    |
| int2float  | 271     | 938.40                 | 886.41       | 0.0084    | 1,068.52               | 1,068.23     | 0.0015    | 829.55                 | 1,045.27     | 0.0110    |
| dec        | 312     | 1,216.17               | 712.31       | 0.0226    | 1,376.32               | 1,257.90     | 0.0292    | 1,376.32               | 1,257.90     | 0.0292    |
| router     | 317     | 1,657.84               | 3,685.60     | 0.0236    | 1,356.30               | 4,086.01     | 0.0201    | 1,177.38               | 3,214.47     | 0.0167    |
| cavlc      | 703     | 2,858.99               | 1,391.17     | 0.0306    | 2,870.25               | 1,718.93     | 0.0467    | 2,312.22               | 1,493.83     | 0.0353    |
| priority   | 1,106   | 6,586.32               | 17,277.00    | 0.0993    | 4,713.27               | 14,011.38    | 0.0693    | 5,268.80               | 17,847.96    | 0.0773    |
| adder      | 1,276   | 7,573.51               | 19,252.33    | 0.0933    | 3,921.26               | 17,057.93    | 0.0511    | 4,128.96               | 18,386.91    | 0.0552    |
| i2c        | 1,489   | 5,615.39               | 1,365.56     | 0.0623    | 5,874.38               | 1,838.38     | 0.0847    | 4,919.72               | 1,405.88     | 0.0681    |
| max        | 3,377   | 18,470.21              | 41,957.09    | 0.2390    | 13,890.82              | 30,964.82    | 0.2150    | 13,346.55              | 28,368.59    | 0.2100    |
| bar        | 3,471   | 10,959.26              | 4,993.18     | 0.1750    | 18,342.59              | 8,062.43     | 0.3260    | 15,162.04              | 7,235.70     | 0.2490    |
| sin        | 5,440   | 43,845.80              | 27,658.14    | 0.9000    | 27,699.06              | 20,565.40    | 0.5240    | 29,926.20              | 30,132.74    | 0.5800    |
| arbiter    | 12,095  | 40,650.23              | 8,641.04     | 0.6710    | 46,036.65              | 8,170.64     | 0.8970    | 38,745.91              | 7,782.34     | 0.7440    |
| voter      | 14,759  | 139,540.08             | 8,000.55     | 2.3200    | 91,849.34              | 7,312.21     | 1.6200    | 93,568.48              | 7,970.67     | 1.6900    |
| square     | 18,548  | 148,728.89             | 20,094.88    | 2.4500    | 88,584.96              | 17,836.24    | 1.4700    | 91,456.46              | 18,908.50    | 1.4800    |
| sqrt       | 24,746  | 204,521.14             | 1,517,812.00 | 4.5300    | 143,030.92             | 789,714.62   | 2.8900    | 130,052.23             | 861,341.06   | 2.6000    |
| multiplier | 27,190  | 204,043.19             | 29,332.91    | 3.9100    | 133,090.14             | 24,757.79    | 2.3800    | 133,267.81             | 28,860.17    | 2.3900    |
| log2       | 32,092  | 260,310.91             | 168,470.03   | 5.2700    | 150,314.16             | 42,295.55    | 2.7700    | 149,226.86             | 49,459.25    | 2.7900    |
| mem_ctrl   | 48,040  | 248,791.11             | 68,602.83    | 4.0700    | 203,206.14             | 24,483.87    | 3.5000    | 173,094.75             | 19,936.07    | 2.9800    |
| div        | 57,375  | 523,522.09             | 690,338.06   | 10.7000   | 280,609.12             | 399,128.59   | 5.1100    | 277,827.69             | 436,291.28   | 5.1700    |
| hyp        | 214,591 | 1,760,069.25           | 5,888,406.50 | 33.2000   | 1,171,167.00           | 4,321,252.00 | 20.3000   | 1,203,100.12           | 4,261,045.00 | 20.1000   |

| Circuit        | Nodes   | LevelSyn-pwmap (Ours)     |                            |                            | LevelSyn-pfmap (Ours)     |                            |                            |
|----------------|---------|---------------------------|----------------------------|----------------------------|---------------------------|----------------------------|----------------------------|
|                |         | Area (Imp. <sup>†</sup> ) | Delay (Imp. <sup>†</sup> ) | Power (Imp. <sup>†</sup> ) | Area (Imp. <sup>‡</sup> ) | Delay (Imp. <sup>‡</sup> ) | Power (Imp. <sup>‡</sup> ) |
| ctrl           | 181     | 601.83 (0.4%)             | 741.25 (-0.5%)             | 0.0088 (1.5%)              | 519.25 (3.7%)             | 799.10 (3.3%)              | 0.0076 (2.3%)              |
| int2float      | 271     | 1,042.25 (2.5%)           | 1,113.13 (-4.2%)           | 0.0151 (0.7%)              | 825.79 (0.5%)             | 1,009.60 (3.4%)            | 0.0109 (0.9%)              |
| dec            | 312     | 1,376.32 (0.0%)           | 1,257.90 (0.0%)            | 0.0283 (3.1%)              | 1,396.34 (-1.5%)          | 1,257.90 (0.0%)            | 0.0295 (-1.0%)             |
| router         | 317     | 1,307.50 (3.6%)           | 4,162.74 (-1.9%)           | 0.0194 (3.5%)              | 1,174.88 (0.2%)           | 3,213.78 (0.0%)            | 0.0167 (0.0%)              |
| cavlc          | 703     | 2,818.95 (1.8%)           | 1,510.57 (12.1%)           | 0.0439 (6.0%)              | 2,324.73 (-0.5%)          | 1,516.30 (-1.5%)           | 0.0354 (-0.3%)             |
| priority       | 1,106   | 4,839.64 (-2.7%)          | 13,963.57 (0.3%)           | 0.0710 (-2.4%)             | 5,637.91 (-7.0%)          | 18,074.93 (-1.3%)          | 0.0826 (-6.9%)             |
| adder          | 1,276   | 3,856.20 (1.7%)           | 17,020.18 (0.2%)           | 0.0493 (3.5%)              | 4,331.65 (-4.9%)          | 16,410.26 (10.8%)          | 0.0609 (-10.3%)            |
| i2c            | 1,489   | 5,873.13 (0.0%)           | 2,293.93 (-24.8%)          | 0.0855 (-0.9%)             | 4,918.47 (0.0%)           | 1,541.01 (-9.6%)           | 0.0681 (0.0%)              |
| max            | 3,377   | 16,429.51 (-18.3%)        | 22,742.97 (26.6%)          | 0.2610 (-21.4%)            | 13,663.10 (-2.4%)         | 20,788.38 (26.7%)          | 0.2110 (-0.5%)             |
| bar            | 3,471   | 13,387.84 (27.0%)         | 5,719.02 (29.1%)           | 0.2460 (24.5%)             | 12,092.85 (20.2%)         | 6,473.61 (10.5%)           | 0.2000 (19.7%)             |
| sin            | 5,440   | 29,171.73 (-5.3%)         | 19,504.01 (5.2%)           | 0.5550 (-5.9%)             | 29,428.22 (1.7%)          | 22,205.18 (26.3%)          | 0.5770 (0.5%)              |
| arbiter        | 12,095  | 45,880.25 (0.3%)          | 8,447.16 (-3.4%)           | 0.8960 (0.1%)              | 40,267.37 (-3.9%)         | 6,493.06 (16.6%)           | 0.6810 (8.5%)              |
| voter          | 14,759  | 95,342.69 (-3.8%)         | 7,585.59 (-3.7%)           | 1.6900 (-4.3%)             | 95,544.13 (-2.1%)         | 7,429.42 (6.8%)            | 1.7300 (-2.4%)             |
| square         | 18,548  | 85,638.38 (3.3%)          | 17,756.67 (0.4%)           | 1.4400 (2.0%)              | 83,546.38 (8.6%)          | 18,270.79 (3.4%)           | 1.3700 (7.4%)              |
| sqrt           | 24,746  | 156,964.28 (-9.7%)        | 739,768.12 (6.3%)          | 3.0600 (-5.9%)             | 158,259.28 (-21.7%)       | 752,540.00 (12.6%)         | 3.1700 (-21.9%)            |
| multiplier     | 27,190  | 134,481.47 (-1.1%)        | 38,750.29 (-56.5%)         | 2.3700 (0.4%)              | 141,235.45 (-6.0%)        | 37,628.16 (-30.4%)         | 2.5900 (-8.4%)             |
| log2           | 32,092  | 155,928.30 (-3.7%)        | 48,510.23 (-14.7%)         | 2.8900 (-4.3%)             | 162,428.28 (-8.8%)        | 50,786.61 (-2.7%)          | 3.0500 (-9.3%)             |
| mem_ctrl       | 48,040  | 204,930.28 (-0.8%)        | 19,050.44 (22.2%)          | 3.4500 (1.4%)              | 175,667.22 (-1.5%)        | 18,280.53 (8.3%)           | 2.9400 (1.3%)              |
| div            | 57,375  | 274,719.72 (2.1%)         | 428,615.28 (-7.4%)         | 5.0200 (1.8%)              | 281,354.84 (-1.3%)        | 464,370.06 (-6.4%)         | 5.2000 (-0.6%)             |
| hyp            | 214,591 | 1,094,646.12 (6.5%)       | 3,133,495.00 (27.5%)       | 18.9000 (6.9%)             | 1,238,242.50 (-2.9%)      | 3,304,790.25 (22.4%)       | 21.8000 (-8.5%)            |
| Avg. / Improv. |         | 5.48e5 (6.52%)            | 1.57e6 (27.48%)            | 9.45 (6.89%)               | 6.19e5 (-2.89%)           | 1.65e6 (22.44%)            | 1.09e1 (-8.45%)            |

<sup>†</sup> Improvement of *LevelSyn-pwmap* relative to *PigMap-power*.

<sup>‡</sup> Improvement of *LevelSyn-pfmap* relative to *PigMap-performance*.

to realistic logic synthesis and physical design constraints. This transfer learning approach not only accelerates the convergence process but also significantly improves the model’s predictive accuracy and robustness when encountering the structural variations inherent in different circuit categories.

We evaluated the convergence behavior of our proposed LA-GNN (level-async) approach against a standard synchronous (sync) updating scheme (5-layer GCN) over 700 epochs. The training results are shown in Figure 4. Both methods exhibit a rapid reduction in training loss during the initial phase. However, the level-async strategy demonstrates a markedly superior optimization trajectory. While the standard sync method settles at a higher training loss of 0.75, the level-async exhibits a sharper initial descent and converges to a significantly lower loss of 0.61.

We visualize the coordinate regression performance in Figure 5 on the largest benchmark circuit, hyp, which comprises 214,591 nodes. The scatter plots compare the GNN-predicted coordinates (left) and the PigMap GiFt [21, 24] initialization coordinates (right) against the true ground-truth placements. While the GiFt initialization results in a highly chaotic and scattered distribution, our LA-GNN effectively suppresses this noise and concentrate the predicted coordinates into a refined and high-density manifold. This focused prediction suggests that the LA-GNN has successfully learned the intrinsic logical connectivity and global center of gravity of the circuit, rather than falling into the high-variance irregularities seen in GiFt. By providing such a stable and cohesive initial layout, our model offers a much more reliable and high-quality starting point

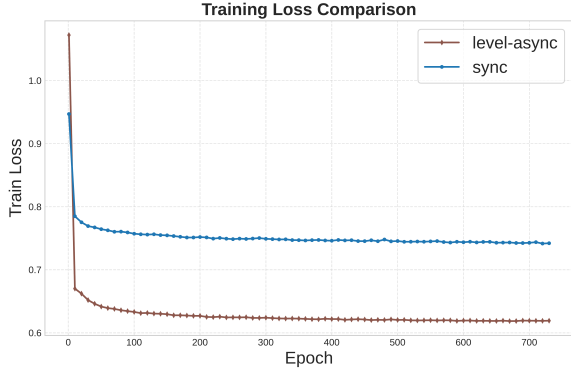


Figure 4: Pre-training loss comparison between LA-GNN and synchronous GNN

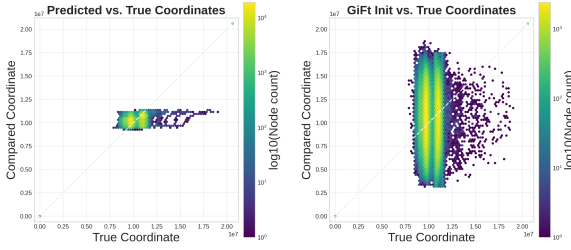


Figure 5: Placement coordinates scatter comparison on hyp. Left: GNN-predicted coordinates. Right: PigMap GiFt[21, 24] initialization coordinates

Table 2: Post-Routing Results on Bar

| Strategies                | Wire Length (um) | Via Counts | DRC Violations |
|---------------------------|------------------|------------|----------------|
| <i>PigMap-power</i>       | 320,689          | 33,963     | 966            |
| <i>PigMap-performance</i> | 301,312          | 28,025     | 623            |
| <i>LevelSyn-pwmap</i>     | 254,439          | 24,330     | 4              |
| <i>LevelSyn-pfmap</i>     | 243,084          | 22,419     | 17             |
| Imp. <sup>†</sup>         | 20.66%           | 28.36%     | 99.59%         |
| Imp. <sup>‡</sup>         | 19.32%           | 20.00%     | 97.27%         |

<sup>†</sup> Improvement of *LevelSyn-pwmap* relative to *PigMap-power*.

<sup>‡</sup> Improvement of *LevelSyn-pfmap* relative to *PigMap-performance*.

for downstream placement optimization, which is crucial for achieving better wirelength and timing convergence in ultra-large-scale designs.

### 4.3 PPA Results

To evaluate the effectiveness of the proposed framework under different optimization goals, we implement two variants of LevelSyn: *LevelSyn-pwmap* and *LevelSyn-pfmap*. The former employs **pwmap** to minimize total wirelength and switching power, while the latter utilizes **pfmap** to focus on critical path delay. Table 1 summarizes the PPA results. LevelSyn consistently outperforms both the logic-only baseline and PigMap across the majority of test cases.

Experimental results demonstrate that LevelSyn consistently outperforms PigMap across different optimization goals: specifically,

*LevelSyn-pwmap* achieves an average 6.52% area reduction and 6.89% switching power improvement compared to *PigMap-power*, with notable gains in large designs like *hyp*. These results demonstrate the accuracy of wirelength estimation provided by LA-GNN. Meanwhile, the delay-oriented *LevelSyn-pfmap* reduces delay by 22.44% by effectively shortening critical paths through Manhattan distance optimization; remarkably, *LevelSyn-pwmap* yields the best overall delay reduction of 27.48%, suggesting that our global spatial anchors and NLI-based positioning offer such high-fidelity spatial guidance that they significantly enhance global structural alignment and timing outcomes beyond traditional logic-depth-based heuristics.

To further evaluate the practical efficacy of our approach within a complete physical design flow, we conduct a comprehensive case study on the Bar circuit, and all strategies are subjected to exactly 20 iterations of routing optimization. The choice of Bar as our primary subject is deliberate: it serves as the central case study in SOTA baseline PigMap.

As summarized in Table 2, our LevelSyn framework demonstrates a decisive advantage over the baseline across all critical metrics. Specifically, when comparing our power-optimized variant (*LevelSyn-pwmap*) to *PigMap-power*, we achieve a 20.66% reduction in total wire length and a 28.36% decrease in via counts. Most notably, our method achieves a near-total elimination of Design Rule Check (DRC) violations, plummeting from 966 to just 4, which is a staggering 99.59% improvement. Similarly, our performance-oriented variant (*LevelSyn-pfmap*) outperforms its counterpart with a 19.32% improvement in wire length and a 97.27% reduction in DRC violations. These results confirm that with the same 20-round optimization budget, the structural stability provided by our LA-GNN allows the router to converge on a significantly more robust, and manufacturable layout.

### 4.4 Runtime Analysis

The runtime analysis is presented in Table 3. In the PigMAP and LevelSyn columns, the parenthetical values represent the breakdown of runtime as (Mapping Time + Pre-processing/Training Time). Specifically, for PigMAP, this includes the GiFt[21] initialization time, whereas for LevelSyn, it accounts for the specific model fine-tuning time. It demonstrates that the LevelSyn approach maintains high efficiency while achieving superior mapping results. Although our methodology includes a comprehensive training strategy, the overall execution time remains highly competitive with established methods like PigMAP.

The empirical data indicates that our specific training approach does not lead to the significant runtime extensions, which is often associated with deep learning applications in EDA. In several cases such as the multiplier and mem\_ctrl circuits, the total runtime of LevelSyn is even comparable to or slightly lower than that of PigMAP. This proves that the fine-tuning phase is remarkably efficient across various circuit scales. Furthermore, while the mapping duration for certain large scale circuits like sqrt and hyp shows a relative increase compared to baseline methods, this latency is negligible when evaluated within the context of the entire electronic design automation flow. In actual industrial practice, the subsequent physical design stages including placement and routing

**Table 3: Runtime Comparison**

| Circuit Names | Nodes  | Baseline (s) | PigMAP-power (s)         | PigMAP-performance (s)   | LevelSyn-pwmap (s)                | LevelSyn-pfmap (s)                |
|---------------|--------|--------------|--------------------------|--------------------------|-----------------------------------|-----------------------------------|
| ctrl          | 181    | 0.25176      | 4.26475 (0.26475+4)      | 4.26010 (0.26010+4)      | 0.34770 (0.29462+0.05308)         | 0.30497 (0.25189+0.05308)         |
| int2float     | 271    | 0.24799      | 3.27121 (0.27121+3)      | 3.28982 (0.28982+3)      | 0.35021 (0.27180+0.07841)         | 0.35164 (0.27323+0.07841)         |
| dec           | 312    | 0.26014      | 3.28175 (0.28175+3)      | 3.28115 (0.28115+3)      | 0.3237 (0.31036+0.01334)          | 0.33829 (0.32495+0.01334)         |
| router        | 317    | 0.25183      | 5.27981 (0.27981+5)      | 5.28016 (0.28016+5)      | 0.43717 (0.28035+0.15682)         | 0.43670 (0.27988+0.15682)         |
| cavlc         | 703    | 0.28075      | 3.32366 (0.32366+3)      | 3.32096 (0.32096+3)      | 0.38326 (0.30978+0.07348)         | 0.40011 (0.32663+0.07348)         |
| priority      | 1106   | 0.26983      | 5.34651 (0.34651+5)      | 5.35716 (0.35716+5)      | 1.33215 (0.35595+0.97620)         | 1.33603 (0.35983+0.97620)         |
| adder         | 1276   | 0.26596      | 5.36133 (0.36133+5)      | 5.36947 (0.36947+5)      | 1.46657 (0.36737+1.09920)         | 1.46161 (0.36241+1.09920)         |
| i2c           | 1489   | 0.28782      | 4.41634 (0.41634+4)      | 4.41147 (0.41147+4)      | 0.51008 (0.42255+0.08753)         | 0.52417 (0.43664+0.08753)         |
| max           | 3377   | 0.31464      | 6.08305 (2.08305+4)      | 5.65394 (1.65394+4)      | 2.81013 (1.68296+1.12717)         | 2.83736 (1.71019+1.12717)         |
| bar           | 3471   | 0.29879      | 4.06734 (1.06734+3)      | 4.04371 (1.04371+3)      | 1.27092 (1.05667+0.21425)         | 1.39967 (1.18542+0.21425)         |
| sin           | 5440   | 0.53017      | 6.58373 (2.58373+4)      | 6.66237 (2.66237+4)      | 3.43735 (2.61093+0.82642)         | 3.60141 (2.77499+0.82642)         |
| arbiter       | 12095  | 0.54331      | 22.62736 (16.62736+6)    | 20.33924 (14.33924+6)    | 16.93909 (16.41279+0.52630)       | 13.79946 (13.27316+0.52630)       |
| voter         | 14759  | 0.80098      | 20.50558 (15.50558+5)    | 20.87620 (15.87620+5)    | 15.82894 (15.43108+0.39786)       | 15.88984 (15.49198+0.39786)       |
| square        | 18548  | 0.95917      | 23.97164 (18.97164+5)    | 24.19507 (19.19507+5)    | 20.14333 (18.48149+1.66184)       | 20.22944 (18.56760+1.66184)       |
| sqrt          | 24746  | 1.11424      | 32.76447 (26.76447+6)    | 33.58574 (27.58574+6)    | 58.05863 (27.36181+30.69682)      | 57.99451 (27.29769+30.69682)      |
| multiplier    | 27190  | 1.31417      | 47.66312 (40.66312+7)    | 47.80954 (40.80954+7)    | 43.68709 (41.51560+2.17149)       | 42.88535 (40.71386+2.17149)       |
| log2          | 32092  | 1.77527      | 60.93639 (52.93639+8)    | 61.99767 (53.99767+8)    | 57.04942 (54.20919+2.84023)       | 57.19582 (54.35559+2.84023)       |
| mem_ctrl      | 48040  | 1.33803      | 167.97727 (158.97727+9)  | 161.89096 (152.89096+9)  | 154.33898 (153.26823+1.07075)     | 152.5755 (151.50475+1.07075)      |
| div           | 57375  | 2.72399      | 290.26830 (281.26830+9)  | 270.12475 (261.12475+9)  | 318.20752 (269.53765+48.66987)    | 289.11868 (240.44881+48.66987)    |
| hyp           | 214591 | 11.33212     | 968.93256 (943.93256+25) | 970.12293 (945.12293+25) | 1,647.60024 (904.50277+743.09747) | 1,726.23648 (983.13901+743.09747) |

**Table 4: Peak GPU Memory Comparison**

typically require several hours or even days to complete. Consequently, the marginal extra time spent during the logic mapping stage is a minor trade-off for the substantial improvements in power and performance metrics provided by LevelSyn. By producing a more optimized netlist, our approach likely eases the burden on downstream back-end tools and facilitates faster timing closure.

#### 4.5 Ablation Study

To evaluate the memory efficiency of our proposed LA-GNN architecture, we conduct an ablation study comparing its peak GPU memory consumption against a 5-layer standard GCN baseline. As summarized in Table 4, the results reveal a significant crossover in memory performance as the circuit scale increases.

Inherently, the LA-GNN architecture requires more memory than a standard GCN due to the additional overhead of maintaining level-specific states and managing asynchronous message-passing buffers. This is evident in several smaller benchmarks where LA-GNN’s allocated memory slightly exceeds that of the GCN. However, this architectural overhead is effectively mitigated by our LASP Strategy as we move toward large-scale designs.

The standard GCN suffers from poor scalability because it requires simultaneous storage of the entire graph’s adjacency structure and gradient states, leading to a memory explosion on industrial-sized circuits. In contrast, our partitioning strategy decomposes massive AIGs into optimized sub-graphs, allowing the model to process level-dependent information without overloading the GPU’s VRAM. For the largest benchmark, hyp, this synergy allows LA-GNN to use only 4,709 MB of allocated memory compared to the GCN’s 8,956 MB, which is a reduction of approximately 47.4%. These results demonstrate that while LAGNN is more complex by design, the integration of our sub-graph partitioning makes it a far more scalable and practical solution for ultra-large-scale circuit placement, where memory constraints are most critical.

| Circuit Name | Nodes  | LA-GNN         |               | GCN            |               |
|--------------|--------|----------------|---------------|----------------|---------------|
|              |        | allocated (MB) | reserved (MB) | allocated (MB) | reserved (MB) |
| ctrl         | 181    | 28.209         | 54.000        | 25.272         | 50.000        |
| int2float    | 271    | 35.948         | 62.000        | 28.490         | 54.000        |
| router       | 312    | 51.848         | 76.000        | 31.605         | 58.000        |
| dec          | 317    | 30.314         | 54.000        | 30.594         | 56.000        |
| cavlc        | 703    | 68.501         | 88.000        | 47.588         | 72.000        |
| priority     | 1106   | 308.150        | 332.000       | 68.157         | 90.000        |
| i2c          | 1276   | 125.399        | 132.000       | 75.263         | 90.000        |
| adder        | 1489   | 391.053        | 406.000       | 82.287         | 110.000       |
| bar          | 3377   | 89.509         | 110.000       | 153.047        | 178.000       |
| max          | 3471   | 291.921        | 336.000       | 173.920        | 220.000       |
| sin          | 5440   | 241.344        | 250.000       | 218.373        | 272.000       |
| arbiter      | 12095  | 299.112        | 604.000       | 547.051        | 616.000       |
| voter        | 14759  | 340.547        | 678.000       | 650.676        | 716.000       |
| square       | 18548  | 431.628        | 890.000       | 815.992        | 880.000       |
| sqrt         | 24746  | 1315.981       | 2080.000      | 923.094        | 1024.000      |
| multiplier   | 27190  | 570.486        | 1080.000      | 1091.725       | 1242.000      |
| log2         | 32092  | 621.266        | 1428.000      | 1200.106       | 1384.000      |
| mem_ctrl     | 48040  | 992.337        | 1792.000      | 1911.500       | 2156.000      |
| div          | 54375  | 1259.288       | 2828.000      | 2258.522       | 2490.000      |
| hyp          | 214591 | 4709.544       | 9604.000      | 8956.849       | 10668.000     |

## 5 Conclusion and Future Work

In conclusion, this paper presents PhySyn, a framework that successfully bridges the gap between logic synthesis and physical implementation by embedding deep learning-based spatial predictions into the synthesis flow. Through the use of LA-GNN and level-aligned partitioning, PhySyn achieves high-fidelity spatial estimation and scalability for large-scale designs. Specifically, the near-total elimination of DRC violations underscores the potential of physical-aware synthesis in achieving correct-by-construction design flows. Moving forward, we plan to extend this framework to support sequential circuits by incorporating flip-flop positioning and to explore multi-objective optimization targets such as congestion and thermal distribution.

## Acknowledgments

This work was supported in part by the Hong Kong Research Grants Council (RGC) under Grant No. 14202824, 14205925, C6003-24Y, and T46-415/25-R.

## References

- [1] Luca Gaetano Amarù, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. The EPFL Combinational Benchmark Suite. <https://api.semanticscholar.org/CorpusID:13971503>
- [2] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*. Springer, 24–40.
- [3] Chau-Shen Chen, Yu-Wen Tsay, TingTing Hwang, A.C.H. Wu, and Youn-Long Lin. 1995. Combining technology mapping and placement for delay-minimization in FPGA designs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (1995).
- [4] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Ru Huang, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, et al. 2024. The dawn of ai-native eda: Opportunities and challenges of large circuit models. *arXiv preprint arXiv:2403.07257* (2024).
- [5] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Ru Huang, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, et al. 2024. Large circuit models: opportunities and challenges. *Science China Information Sciences* 67, 10 (2024), 200402.
- [6] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 257–266.
- [7] Chenhui Deng, Zichao Yue, Cunxi Yu, Gokce Sarar, Ryan Carey, Rajeev Jain, and Zhiru Zhang. 2024. Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [8] Wenji Fang, Wenkai Li, Shang Liu, Yao Lu, Hongce Zhang, and Zhiyao Xie. 2025. NetTAG: A Multimodal RTL-and-Layout-Aligned Netlist Foundation Model via Text-Attributed Graph. *arXiv preprint arXiv:2504.09260* (2025).
- [9] Wenji Fang, Shang Liu, Jing Wang, and Zhiyao Xie. 2025. CircuitFusion: Multimodal Circuit Representation Learning for Agile Chip Design. In *International Conference on Learning Representations (ICLR)*.
- [10] Tong Gao, Kuang-Chien Chen, J. Cong, Yuzheng Ding, and C. Liu. 1993. Placement and placement driven technology mapping for FPGA synthesis. In *Sixth Annual IEEE International ASIC Conference and Exhibit*.
- [11] W. Gosti, S.R. Khatri, and A.L. Sangiovanni-Vincentelli. 2001. Addressing the timing closure problem by integrating logic optimization and placement. In *IEEE/ACM International Conference on Computer Aided Design. ICCAD 2001. IEEE/ACM Digest of Technical Papers (Cat. No.01CH37281)*.
- [12] Mingxiao He, Pengcheng Huang, Zhenyu Zhao, and Peiyun Bian. 2025. CPA-Remap: Critical-Path-Based Physically Aware Remapping Framework for Timing Optimization. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 393–400.
- [13] R. Ho, K.W. Mai, and M.A. Horowitz. 2001. The future of wires. *Proc. IEEE* 89, 4 (2001), 490–504.
- [14] Guyue Huang, Jingbo Hu, Yifan He, Jialong Liu, Mingyuan Ma, Zhaoyang Shen, Juejian Wu, Yuanfan Xu, Hengrui Zhang, Kai Zhong, Xuefei Ning, Yuzhe Ma, Haoyu Yang, Bei Yu, Huazhong Yang, and Yu Wang. 2021. Machine learning for electronic design automation: A survey. *ACM Trans. Des. Autom. Electron. Syst.* (2021).
- [15] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. (Dec. 1998), 359–392.
- [16] Cunqing Lan, Hongyang Pan, Zhiang Wang, Xuan Zeng, Fan Yang, and Keren Zhu. 2025. PigMap2: A Physical Information Guided Technology Mapping Framework. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [17] Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. 2022. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 667–672.
- [18] Min Li, Zhengyuan Shi, Qiuxia Lai, Sadaf Khan, Shaowei Cai, and Qiang Xu. 2023. On eda-driven learning for sat solving. In *Proceedings of the 60th Annual ACM/IEEE Design Automation Conference*.
- [19] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Brucec Khailany, and David Z. Pan. 2021. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2021).
- [20] Jiawei Liu, Jianwang Zhai, Mingyu Zhao, Zhe Lin, Bei Yu, and Chuan Shi. 2024. PolarGate: Breaking the Functionality Representation Bottleneck of And-Inverter Graph Neural Network. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*.
- [21] Yiting Liu, Hai Zhou, Jia Wang, Fan Yang, Xuan Zeng, and Li Shang. 2025. The Power of Graph Signal Processing for Chip Placement Acceleration. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*. Association for Computing Machinery.
- [22] Aiguo Lu, Hans Eisenmann, Guenter Stenz, and Frank M Johannes. 1998. Combining technology mapping with post-placement resynthesis for performance optimization. In *Proceedings International Conference on Computer Design. VLSI in Computers and Processors (Cat. No. 98CB36273)*. IEEE, 616–621.
- [23] Alan Mishchenko et al. 2007. ABC: A system for sequential synthesis and verification. URL <http://www.eecs.berkeley.edu/alanmi/abc> 17 (2007).
- [24] Hongyang Pan, Cunqing Lan, Yiting Liu, Zhiang Wang, Li Shang, Xuan Zeng, Fan Yang, and Keren Zhu. 2025. Physically Aware Synthesis Revisited: Guiding Technology Mapping with Primitive Logic Gate Placement. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. Article 171.
- [25] Massoud Pedram and Narasimha Bhat. 1991. Layout driven technology mapping. In *Proceedings of the 28th ACM/IEEE Design Automation Conference*. 99–105.
- [26] André Inácio Reis and Jody M. A. Matos. [n. d.]. *Physical Awareness Starting at Technology-Independent Logic Synthesis*.
- [27] Amir H. Salek, Jinan Lou, and Massoud Pedram. 1998. A DSM design flow: putting floorplanning, technology-mapping, and gate-placement together. In *Proceedings of the 35th Annual Design Automation Conference (DAC '98)*.
- [28] Zhengyuan Shi, Min Li, Sadaf Khan, Liuzheng Wang, Naixing Wang, and Yu Huang. 2022. DeepTPI: Test Point Insertion with Deep Reinforcement Learning. In *2022 IEEE International Test Conference (ITC)*.
- [29] Zhengyuan Shi, Hongyang Pan, Sadaf Khan, Min Li, Yi Liu, Junhua Huang, Hui-Ling Zhen, Mingxuan Yuan, Zhufei Chu, and Qiang Xu. 2023. DeepGate2: Functionality-Aware Circuit Representation Learning. In *Proceedings of the 2023 IEEE/ACM international conference on Computer-aided design*.
- [30] SkyWater Technology and Google. 2020. SkyWater Open Source PDK. <https://github.com/google/skywater-pdk> Accessed: 2026-04-05.
- [31] The OpenROAD Project. 2024. OpenROAD: An open-source industrial design flow from RTL to GDS. <https://github.com/The-OpenROAD-Project/OpenROAD>.
- [32] N. Togawa, M. Sato, and T. Ohtsuki. 1994. Maple: A Simultaneous Technology Mapping, Placement, And Global Routing Algorithm For Field-programmable Gate Arrays. In *IEEE/ACM International Conference on Computer-Aided Design*. 156–163. doi:10.1109/ICCAD.1994.629759
- [33] N. Togawa, M. Yanagisawa, and T. Ohtsuki. 1998. Maple-opt: a performance-oriented simultaneous technology mapping, placement, and global routing algorithm for FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (1998), 803–818. doi:10.1109/43.720317
- [34] Clifford Wolf, Johann Glaser, and Johannes Kepler. 2013. Yosys-a free Verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*. 97.
- [35] Xinyue Wu, Zixuan Li, Fan Hu, Ting Lin, Xiaotian Zhao, Runxi Wang, and Xinfei Guo. 2025. Shift-Left Techniques in Electronic Design Automation: A Survey. *arXiv preprint arXiv:2509.14551* (2025).
- [36] Zhengfeng Wu, Pratik Shrestha, Saran Phatharodom, and Ioannis Savidis. 2025. Differentiable Graph Neural Networks for Wavelength Estimation. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [37] Zhiyao Xie, Rongjian Liang, Xiaoqing Xu, Jiang Hu, Yixiao Duan, and Yiran Chen. 2021. Net2: A graph attention network method customized for pre-placement net length estimation. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference*. 671–677.